

Benchmark Stream Construction, Not Attack Prevalence, Produces the Regime Structure of Streaming Intrusion Detection

ANONYMOUS AUTHOR(S)

Streaming intrusion-detection results are almost always reported on benchmark streams that were *assembled*: temporally disjoint captures pooled by round robin, or capture days interleaved to balance splits. We show that this assembly step, not attack prevalence, produces the regime structure the literature reports. On CICIDS2017, holding the record multiset identical and changing only the ordering moves held-out prevalence by 42.9954 percentage points (68.235% to 25.2396%), leaves the two held-out slices sharing only 32.5% of their records, and inverts the ordering of the two deterministic methods under test: a plain batch baseline (ECOD) beats the evaluated detector under true timestamp order and loses to it under the interleaving. On LITNET-2020, the composite’s single 6.4982% operating point is an identity of pooling: the three temporally disjoint captures span 0.176% to 15.7747% held-out prevalence individually, and the pooled figure is their equal-weight mean. We further characterize the evaluated detector itself: its run-length posterior is algebraically pinned to the hazard rate below the run-length cap, so the system is prequential global-Gaussian tail scoring rather than change-point detection, and the textbook repair saturates the score and detects nothing on the stream tested. Every number in this paper traces to an archived, hash-verified run manifest, and the claim ledger mapping each introductory claim to its generating run is part of the released artifact.

CCS Concepts: • **Security and privacy** → **Intrusion detection systems**; • **Computing methodologies** → *Machine learning*.

Additional Key Words and Phrases: intrusion detection, streaming evaluation, benchmark construction, attack prevalence, reproducibility, provenance

1 Introduction

Machine-learning evaluations for network intrusion detection rest on a small number of public benchmarks, and a growing body of work questions what those benchmarks measure [2, 7, 21]. This paper adds a specific, mechanically verifiable item to that list: *the step that assembles capture files into an evaluation stream*. Public captures are rarely usable as-is — CICIDS2017 spans five separate days with wildly different attack densities [11, 22], and LITNET-2020’s captures are temporally disjoint [8] — so evaluations interleave days, pool attack types, or replay records round-robin to obtain a single stream with balanced splits. That construction step is usually treated as neutral plumbing. We show it is not: on the same records, it moves the held-out attack prevalence by tens of points, replaces two-thirds of the evaluated sample, and inverts which of the two deterministic methods under test wins.

Origin of this version

Earlier versions of this manuscript reported results produced under the composite construction described above and, in addition, described a scoring rule that the released code did not implement. An independent adversarial review and a subsequent line-by-line audit of the archived artifacts established both points. The present version is a rebuild from that audit: the composite construction is retained only as an explicitly labeled synthetic protocol and contrasted against natural-order evaluation; the detector is described as what the code computes rather than what the prose intended; and every number in this paper was regenerated — or, for the retained prevalence sweep, re-derived from the archived pre-audit runs — by a script that wrote, in the same execution, an archived manifest recording the git commit, configuration hash, input SHA-256, seed, environment hash, and output path. A build gate fails on any number without such a manifest. The correction history, including eighteen numbered corrected incidents, ships with the artifact.

Contributions

- (1) **The construction contrast** (Section 5). On CICIDS2017 we evaluate the identical 1600000-record multiset (352962 attacks; whole-stream prevalence identical in both arms by permutation, verified mechanically) in true timestamp order and in day-of-week round robin. Ordering alone moves held-out prevalence from 68.235% to 25.2396%, leaves the held-out slices sharing 78000 of 240000 records (32.5%), and inverts the deterministic ECOD-versus-detector ordering. The ranking change is a rotation (Kendall $\tau = -0.3333$), not a full reversal.
- (2) **The pooling identity on LITNET-2020** (Section 5.4). The pooled composite’s 6.4982% held-out prevalence equals the equal-weight mean over three captures spanning 0.176% to 15.7747% held-out prevalence; it is a property of the assembly, not of any capture.
- (3) **Method identity** (Section 3). The evaluated detector’s run-length posterior equals the hazard rate for any data — the reset and growth branches share a predictive term that cancels — so the system is sequential global-Gaussian tail scoring with a change-point auxiliary that contributes a mean score of 0.002531 on the 75.039% of records where it binds. The textbook repair (a vague prior predictive on the reset branch) makes the posterior respond but saturates the composed score (0.9274 of scores at exactly the cap), detecting nothing on the one stream and prefix tested: both variants are degenerate, in opposite directions.
- (4) **A provenance discipline** (Section 8). Every number resolves through a generated macro layer backed by archived manifests; the build fails on orphans, on macros claimed by two runs with different values, and on drift between the macro index and the manifests behind it.

Two findings that cut against the evaluated detector are stated with the same prominence as the rest: on the identical held-out slice of the interleaved (unresampled) stream, a batch LOF reference scores 0.863194 AUC-PR against the detector’s 0.544998 (Section 6); and the detector’s lift over the chance floor falls from +0.34821 at 10% prevalence to −0.020445 at 64% — below what a constant predictor achieves (Section 6).

2 Related Work

Benchmark criticism. Catillo et al. [7] ask whether results on public intrusion datasets represent concrete advances; Arp et al. [2] catalogue recurring methodological pitfalls, including sampling bias and inappropriate baselines; Ring et al. [21] survey dataset limitations. Engelen et al. [11] correct label and flow-construction defects in CICIDS2017; we build on their corrected release. Our contribution is orthogonal to label quality: we hold the records fixed and vary only the assembly of records into a stream.

Streaming anomaly detection. The baselines evaluated in this rebuild are Half-Space Trees [23] and LODA [20] (streaming), with LOF [5] and ECOD [17] as batch references. KitNET [19], xStream [18], RRCF [13], streaming isolation forest [9], and COPOD [16] have implementations in the artifact but no archived evaluation runs in this rebuild, and carry no numbers here. Cao et al. [6] benchmark streaming detectors under controlled anomaly types and drifts and report strong sensitivity to stream composition; our results give that sensitivity a mechanism on real IDS benchmarks. Only the baselines with archived runs in this rebuild (HST, LODA, ECOD, LOF) carry numbers in this paper.

Change-point detection. The evaluated detector descends from Bayesian online change-point detection [1] with a truncated run-length posterior [12, 14]. Section 3 shows the implementation’s posterior is data-independent, which places the evaluated system outside this family despite its ancestry; van den Burg and Williams [24] document how often change-point evaluations mismeasure their subject.

Conformal and calibrated alerting. Laxhammar and Falkman [15] established conformal anomaly detection for sequential data, with guarantees inherited from exchangeability assumptions [25]; FADES [3] applies conformal signals to drift estimation for streaming IDS. Our findings constrain this line: constructions that reorder records to balance splits manufacture the exchangeability those methods assume, while natural-order streams violate it conspicuously (Section 4).

Cost-sensitive thresholds. The threshold actually implemented is the prior-inclusive Bayes rule of Elkan [10] (Section 3); SRE-style error budgets [4] motivated the original framing.

3 The System Under Study

We characterize the detector as implemented, from measurement rather than intention. All measurements in this section are archived under run `s3_score_threshold_verification`.

3.1 The score

The published description defines the anomaly score as the run-length-reset posterior $P(r_t = 0 \mid x_{1:t})$. The implementation returns

$$s_t = \max(\text{tail}_t, 0.25 \cdot P(r_t \leq 5 \mid x_{1:t})),$$

where tail_t is a χ^2 CDF of the squared standardized residual under a global, slowly adapting diagonal Gaussian. Because $0.25 \cdot P(r \leq 5) \leq 0.25$, the change-point branch can set the score only where the tail term is below 0.25. Measured over 49970 post-warm-up records of the natural-order CICIDS2017 stream, the change-point branch binds on 75.039% of records — but at a mean contributed score of 0.002531, so on those records both terms are near zero and the branch supplies a floor, not signal. The discriminative work is done by the tail term throughout. We therefore name the system *prequential global-Gaussian tail scoring with a downweighted change-point auxiliary*.

3.2 The run-length posterior is data-independent

In the implementation, the reset branch and the growth branch share the same run-conditional predictive likelihood, which cancels in the posterior normalization. Consequently $P(r_t = 0)$ equals the hazard rate h exactly, for any data. Under a 6σ mean shift (hazard 0.001): mean $P(r=0)$ before the shift 0.001; peak in the 50-record window after the shift 0.001; maximum deviation from the hazard between initialization and truncation 0 (zero at the recorded precision). Two regimes deviate, and neither is a detection: the first record ($P(r=0) = 1$ trivially, array of length one), and every $t \geq 500$ (the run-length cap), where truncation removes growth mass before normalization and the posterior wanders (mean 0.01099, max 1). Every stream evaluated in this paper is far longer than 500 records, so published runs spent nearly their whole length in the truncated regime.

3.3 The threshold

The implemented decision threshold is the prior-inclusive Bayes rule [10]: $\tau = C_{FP}(1 - \rho) / [C_{FP}(1 - \rho) + C_{FN}\rho]$, giving 0.655172 at incident prior $\rho=0.05$ and 0.261745 at $\rho=0.22$ — not the cost-only $C_{FP}/(C_{FP}+C_{FN}) = 0.090909$ described in earlier versions. AUC-PR and AUC-ROC are threshold-free and unaffected; we do not report threshold-dependent columns, which are additionally non-comparable across methods here (the baselines receive validation-tuned thresholds).

Table 1. The evaluation streams in natural order (Stage 1 manifests). Run lengths are contiguous attack runs in the labeled stream; span is the wall-clock extent of the capture.

stream	records	prevalence	held-out prev.	attack-run med/p90/max	span (min)
CICIDS2017 (week)	1600000	22.0601%	68.235%	2/70/2522	6246.71
LITNET udp_flood	500000	14.9384%	15.7747%	1/2/20	3.97
LITNET blaster_worm	500000	0.6562%	3.544%	1/2/5	1.62
LITNET spam	500000	0.0276%	0.176%	1/1/2	39122.23

3.4 Detection delay, not latency

The quantity previously reported as “latency in milliseconds” is a count of records between attack onset and first alert; the evaluation contains no wall-clock instrumentation (0 clock calls in the latency module). We report it as detection delay in records, and we do not report per-flow compute cost, which this pipeline does not measure.

4 Datasets and Stream Construction

4.1 CICIDS2017: a budgeted subsample of a capture week

We use the Engelen-corrected release [11]. The evaluation dataset is 1600000 of 2087997 eligible records (76.628%), selected by fixed per-day budgets with proportional stride subsampling. Because the budgets are fixed rather than a common factor, larger days are cut harder, and the attack-heavy days are the largest: per-day retention runs from 64.399% (Friday, the densest day) to 93.156% (Tuesday). Table 1 summarizes all four streams. Measured consequence: the eligible week’s prevalence is 24.2065% while the dataset’s is 22.0601%, a bias of -2.1464 points from the budget step alone. The often-quoted “22%” regime is therefore *doubly* constructed: a subsampling budget removed prevalence, then interleaving redistributed what remained. The construction contrast in Section 5 is unaffected — both arms hold the identical multiset — but absolute prevalence figures carry this qualification.

4.2 LITNET-2020: three disjoint captures, no global chronology

LITNET-2020’s captures are temporally disjoint: udp_flood spans roughly four minutes, blaster_worm under two minutes, and spam about four weeks. No coherent global timestamp order exists across them, so we evaluate three per-attack-type streams of 500000 records each, never a manufactured composite; the composite appears only as the explicitly labeled synthetic arm of Section 5.4. Burn-rate-style multi-window alerting (60/360/4320-minute windows) is undefined on captures spanning minutes; we report that as a finding about the benchmark, and no burn-rate results appear in this paper.

4.3 Natural order is not exchangeable

In true timestamp order, CICIDS2017’s held-out tail is a single 204.2-minute Friday-evening window at 68.235% prevalence, and attacks arrive in runs (median run length 2, 90th percentile 70, maximum 2522 on CICIDS2017; essentially isolated single flows on LITNET; Table 1). Among these datasets, in natural order, *no moderate-prevalence regime exists*: held-out prevalence is either high (68.235%) or spans 0.176%–15.7747% per capture. The moderate regimes the literature reports on these benchmarks are products of the constructions examined next.

Table 2. The construction contrast. Held-out prevalence, attacks, and the proposed detector’s AUC-PR per arm. Every cell is a macro resolving to an archived manifest.

Benchmark	Construction	Held-out prev.	Attacks	AUC-PR (proposed)
CICIDS2017	natural (timestamp)	68.235%	163764	0.728337
CICIDS2017	synthetic (day RR)	25.2396%	60575	0.544998
LITNET udp flood	natural (per type)	15.7747%	11831	0.394428
LITNET blaster worm	natural (per type)	3.544%	2658	0.964091
LITNET spam	natural (per type)	0.176%	132	0.846154
LITNET pooled	synthetic (3-type RR)	6.4982%	14621	0.943056

5 The Construction Contrast

5.1 Design

CICIDS2017 admits a clean manipulation: the same record multiset can be presented in true timestamp order (*natural*) or in day-of-week round robin (*synthetic*; the construction used by earlier versions of this work). The reordering is verified mechanically to be an exact permutation: both arms hold 1600000 records and 352962 attacks (22.060125%). Order is the only manipulated variable; the 70/15/15 chronological split rule is held fixed, and no split-rule sensitivity analysis has been run, so effects are attributable to *reordering under this split rule*, not to construction in some larger causal sense. The synthetic arm reproduces the archived earlier evaluation exactly (held-out 60575 attacks in 240000 records), so the contrast is against the construction actually used, not a strawman.

5.2 What reordering does to the evaluated sample

Table 2 summarizes the arms. Held-out prevalence moves by -42.9954 points. The two held-out slices share only 78000 of 240000 records (32.5%); the synthetic arm’s held-out attacks are a strict subset of the natural arm’s, with 103189 attacks relocated into training by the reordering. The mechanism is *dilution*: every synthetic held-out attack is a Friday record (1 of five days contribute attacks), and Monday–Thursday contribute 162000 attack-free held-out rows. The synthetic arm’s Friday sub-slice is denser (77.66%) than the natural arm’s entire slice — the prevalence falls because attack-free days are mixed in, not because attacks are redistributed.

5.3 What reordering does to method comparison

AUC-PR’s chance floor equals test prevalence, which is precisely what the construction moves (0.68235 natural versus 0.252396 synthetic), so raw AUC-PR is not comparable across arms and we do not report cross-arm deltas. Within arms:

- **Natural order:** ECOD 0.755142 > detector 0.728337 (both deterministic). Against the 0.68235 floor, neither clears +0.072792; these values are not a performance claim. HST scores 0.588902 here — 0.093448 *below* chance — from a single seed, so no placement relative to the other methods is asserted for it.
- **Synthetic:** detector 0.544998 > ECOD 0.418966 (both deterministic). Against the 0.252396 floor, every method’s lift is *higher* than in the natural arm. HST’s three-seed values here are mean 0.433044 ± 0.077736 ; its ordering against ECOD flips across those seeds, so it carries no placement.

The ordering of the two deterministic methods inverts: ECOD beats the detector under natural order and loses to it under the synthetic construction, on identical records. This comparison is seed-free — both methods are deterministic here — and it is the paper’s central evidence. Computed over all three methods at the seeds run, the ranking change is

Table 3. LITNET-2020: AUC-PR per stream against the pooled synthetic composite. The detector and ECOD are deterministic; HST is a single draw per cell here, so no ordering involving HST is asserted (its measured composite spread across three seeds is 0.2614 ± 0.097068).

stream	held-out prev.	detector	ECOD	HST (1 seed)
udp_flood (natural)	15.7747%	0.394428	0.321185	0.552675
blaster_worm (natural)	3.544%	0.964091	0.03571	0.147731
spam (natural)	0.176%	0.846154	0.106805	0.008447
pooled (synthetic)	6.4982%	0.943056	0.229143	0.23884

a rotation, not a reversal (Kendall $\tau = -0.3333$, with 1 of 3 pairwise orderings preserved), but any placement of HST within it is single-draw here, and Section 5.5 reports how it moves with the seed. One caveat binds ECOD’s reading throughout: ECOD is fitted on benign-only training data and is therefore label-privileged relative to the unsupervised streaming methods; where ECOD wins, label supervision is an uncontrolled explanation.

5.4 LITNET-2020: pooling as construction

Round robin *within* a per-attack-type stream is the identity, so the LITNET contrast is compositional: three natural per-type streams against the pooled three-type composite used by earlier versions. The pooled held-out slice is, by construction, exactly the union of the three per-type held-out slices (equal budgets, a perfect three-cycle, and a divisible split boundary), so its prevalence 6.4982% is the equal-weight mean of 15.7747%, 3.544%, and 0.176% — an identity, not a measurement, and the equal weighting is an artifact of equal capture budgets. The composite reports one operating point for three populations that share none. Per-stream results (deterministic methods): the detector scores 0.964091 on blaster_worm and 0.846154 on spam against ECOD’s 0.03571 and 0.106805; on udp_flood the detector scores 0.394428 against ECOD’s 0.321185. On the pooled composite the detector scores 0.943056 against ECOD’s 0.229143: pooling collapses three heterogeneous held-out regimes into one operating point and erases the per-stream structure the table shows (Table 3).

5.5 Seed sensitivity of stochastic placements

HST is the only stochastic method in this contrast. Its placement flips with the seed in both cells where we bought additional seeds: on the LITNET composite, HST across seeds 11/23/47 is 0.2614 ± 0.097068 AUC-PR against deterministic ECOD 0.229143 — above ECOD at two seeds, below at the third; on CICIDS synthetic the analogous ordering also flips across three seeds. Accordingly, *no ranking count and no per-stream ranking attribution involving HST is stated in this paper*; HST values appear only with their seed distributions, and flat comparative claims are made between deterministic methods only.

6 Prevalence Within the Synthetic Construction

An earlier version of this work reported a prevalence sweep as evidence about deployment regimes. Retained here, it is a controlled experiment *on the interleaved synthetic construction* and nothing more: prevalence is varied by stratified resampling of the interleaved CICIDS2017 stream at five levels, three resampling draws each. Table 4 reports AUC-PR with the achieved held-out prevalence as the chance floor beside every level; only at the unresampled level do all draws see the same records.

Table 4. AUC-PR by prevalence level on the interleaved synthetic construction (mean over three resampling draws). “Floor” is the achieved held-out prevalence. At the unresampled level, deterministic-method cells are identical across draws; at resampled levels the small spread of every method is the resampling draw. Per-draw values are in the archived findings.

Level	Floor	proposed detector	HST	LODA	ECOD (batch)	LOF (batch)
5%	0.049524	0.367164	0.199261	0.207581	0.166239	0.537596
10%	0.099004	0.447213	0.299702	0.247426	0.252198	0.697003
unresampled	0.252396	0.544998	0.433044	0.341715	0.418966	0.863194
40%	0.399995	0.494217	0.476031	0.399422	0.527364	0.896968
64%	0.639991	0.619546	0.499341	0.524691	0.678404	0.904855

Table 5. The repair saturates the score. Score-distribution diagnostics for both variants over the first 15000 records of the ablation stream, and detection metrics on the 200000-record prefix (held-out 30000 records, 4840 attacks, chance floor 0.161333).

	evaluated detector	corrected statistic
mean posterior mass $P(r \leq 5)$	0.006472	1
scores exactly at the 0.25 cap	0.004733	0.9274
distinct score values	3899	747
score standard deviation	0.218999	0.159341
AUC-PR on the prefix	0.397486	0.169912
AUC-ROC on the prefix	0.848216	0.50962

Two findings, both deterministic-versus-deterministic at the unresampled level and therefore stated flatly. First, the batch LOF reference dominates the detector at every level — at the unresampled level, 0.863194 against 0.544998 on the identical held-out slice. Second, the detector’s lift over the chance floor runs +0.31764 at 5%, peaks at +0.34821 at 10%, and falls from there to −0.020445 at 64% — below the floor a constant predictor achieves. The “low-prevalence advantage” reported earlier is a lift-versus-prevalence gradient inside one synthetic construction; nothing in this section transfers to natural order (Section 5 measures the same detector under a different ordering and finds a different result), and nothing here is a claim about deployment prevalence.

7 Repairing the Change-Point Statistic

Section 3 showed the evaluated posterior never resets. The textbook repair is a prior-predictive term on the reset branch. We implemented it (a Normal–Inverse-Gamma prior predictive: Student- t , $\nu=2$, squared scale twice the global variance, standard vague hyperparameters, nothing tuned) and measured both variants under a 30-minute compute cap whose reductions bind this paragraph’s scope: one stream (udp_flood), a 200000-record prefix, one seed (both variants are deterministic). On the synthetic probe the repair works: $P(r=0)$ peaks at 1 after a 6σ shift (1000× the hazard) where the original stays at 0.001. On real data it detects nothing: AUC-PR 0.169912 against the original’s 0.397486 (floor 0.161333), with AUC-ROC 0.50962 — chance. The diagnostic explains why, and why “the repair degrades detection” is the wrong reading: under the repair the posterior collapses onto short runs at every step (mean $P(r \leq 5) = 1$), the $0.25 \cdot P(r \leq 5)$ branch saturates, 0.9274 of scores sit exactly at the 0.25 cap, and only 747 distinct values are emitted against the original’s 3899. A score constant on most records cannot rank; an AUC-ROC of 0.50962 means constant, not worse (Table 5).

Both variants are therefore degenerate in opposite directions — the evaluated one never resets (branch cancellation), the repaired one always resets (a $v=2$ prior predictive prefers a fresh run for nearly every point) — and neither is change-point detection. We draw no implication about what a tuned prior-predictive scale would achieve: locating one is a hyperparameter search this budget excludes, and selecting it on test labels is forbidden by our protocol.

8 Provenance Discipline

Every number in this paper is a $\LaTeX$ macro generated from archived run manifests recording git commit, configuration hash, input SHA-256, seed, environment hash, timestamps, and output paths. A gate fails the build if a manuscript number lacks a manifest, if two manifests claim one macro with different values, or if the derived macro index disagrees with the manifests behind it. A claim ledger maps every sentence of the abstract and introduction to its generating run. The discipline exists because it caught real defects during this rebuild — among them a smoke-test manifest shadowing a full run, a manuscript-bound table cell backed by no manifest, and a derived index that had silently lost every headline macro — each recorded as a numbered corrected incident in the released artifact.

9 Threats to Validity

Internal. ECOD is fitted on benign-only training data and is label-privileged; where it wins, supervision is an uncontrolled explanation. The detector uses a fixed untuned threshold while baselines receive validation-tuned thresholds; we therefore report only threshold-free metrics. Stochastic methods rest on one seed except where seed distributions are reported, and every flat claim in this paper is between deterministic methods.

External. Two benchmarks, one split rule. The construction effect is measured under a fixed 70/15/15 chronological split; no split-rule sensitivity analysis exists, and the CICIDS effect operates through per-day attack scheduling meeting a fractional tail split. The CICIDS dataset is a 76.628% budgeted subsample with a measured -2.1464-point prevalence bias. The natural-order held-out slice is one 204.2-minute window; that is what true chronology on this benchmark yields, and it is stated rather than engineered around. The Stage-6 ablation covers one stream and one prefix. Nothing here estimates deployment prevalence or deployment performance.

Construct. We do not claim the evaluated detector is change-point detection (Section 3), and we do not claim the repaired variant establishes what corrected change-point detection would achieve (Section 7).

10 Conclusion

On these benchmarks, the regime structure that motivates streaming IDS evaluations — rare-attack regimes, moderate regimes, regime-dependent method rankings — is manufactured by the step that assembles captures into streams. The same records, reordered, move held-out prevalence by 42.9954 points (68.235% to 25.2396%), replace two-thirds of the evaluated sample, and invert which of the two deterministic methods wins. Pooling disjoint captures reports a prevalence no capture exhibits. Evaluations that interleave or pool should either report the natural-order result beside the constructed one, as this paper does, or state that their operating point is a property of the assembly. The corrected artifacts, manifests, and claim ledger accompany this paper so that every number can be traced to the run that produced it.

Data and Artifact Availability

The code, stream-construction scripts with SHA-256 verification, all run manifests, the macro layer, the claim ledger, and the corrected-incident history are archived and will be linked here upon acceptance. During double-anonymous review, the artifact is available through the submission system’s anonymous artifact channel.

Statement on Generative AI Usage

Large language models were used, under continuous human direction, for engineering assistance in this work: writing and refactoring analysis scripts, drafting prose that was subsequently verified against the archived measurements, and operating the adversarial review harness that audited the authors’ own claims. Every number in this paper was produced by executed code with an archived manifest, not by a language model; the claim ledger and provenance gate exist in part to make that boundary auditable. The authors take full responsibility for all content.

Companion Paper Disclosure

A related manuscript by the same research programme (reference suppressed for double-anonymous review) shares parts of the underlying codebase and predates the audit that produced the present version. The measurements reported here concerning the implemented scoring rule (Section 3) apply to that shared lineage; a correction process for the companion manuscript is in progress and has been disclosed to the relevant editors. The two papers report disjoint result sets.

References

- [1] Ryan Prescott Adams and David J. C. MacKay. 2007. Bayesian Online Changepoint Detection. *arXiv preprint arXiv:0710.3742* (2007).
- [2] Daniel Arp, Erwin Quiring, Feargus Pendlebury, Alexander Warnecke, Fabio Pierazzi, Christian Wressneger, Lorenzo Cavallaro, and Konrad Rieck. 2022. Dos and Don’ts of Machine Learning in Computer Security. In *31st USENIX Security Symposium*. 3971–3988.
- [3] Seth Barrett, Gokila Dorai, Lin Li, and Swarnamugi Rajaganapathy. 2026. FADES: Adaptive Drift Estimation via Conformal Signals for Streaming Intrusion Detection. *Electronics* 15, 10 (2026), 2114. doi:10.3390/electronics15102114
- [4] Betsy Beyer, Chris Jones, Jennifer Petoff, and Niall Richard Murphy. 2016. *Site Reliability Engineering*. O’Reilly Media.
- [5] Markus M. Breunig, Hans-Peter Kriegel, Raymond T. Ng, and Jorg Sander. 2000. LOF: Identifying Density-Based Local Outliers. In *SIGMOD*.
- [6] Yang Cao, Yixiao Ma, Ye Zhu, and Kai Ming Ting. 2024. Revisiting Streaming Anomaly Detection: Benchmark and Evaluation. *Artificial Intelligence Review* 58 (2024). doi:10.1007/s10462-024-10995-w
- [7] Marta Catillo, Antonio Pecchia, and Umberto Villano. 2023. Machine Learning on Public Intrusion Datasets: Academic Hype or Concrete Advances in NIDS?. In *53rd Annual IEEE/IFIP International Conference on Dependable Systems and Networks – Supplemental Volume (DSN-S)*. 132–136. doi:10.1109/DSN-S58398.2023.00038
- [8] Robertas Damasevicius et al. 2020. LITNET-2020: An Annotated Real-World Network Flow Dataset for Network Intrusion Detection. *Electronics* 9, 5 (2020), 800.
- [9] Zhiguo Ding and Minrui Fei. 2013. An anomaly detection approach based on isolation forest algorithm for streaming data using sliding window. *IFAC Proceedings Volumes* (2013).
- [10] Charles Elkan. 2001. The Foundations of Cost-Sensitive Learning. In *IJCAI*.
- [11] Gints Engelen, Vera Rimmer, and Wouter Joosen. 2021. Troubleshooting an Intrusion Detection Dataset: The CICIDS2017 Case Study. In *IEEE Security and Privacy Workshops*.
- [12] Paul Fearnhead and Zhen Liu. 2007. On-line inference for multiple changepoint problems. *Journal of the Royal Statistical Society: Series B* (2007).
- [13] Sudipto Guha, Nina Mishra, Gourav Roy, and Okke Schrijvers. 2016. Robust Random Cut Forest Based Anomaly Detection on Streams. In *ICML*.
- [14] Jeremias Knoblauch and Theodoros Damoulas. 2018. Spatio-temporal Bayesian Online Changepoint Detection with Model Selection. *arXiv preprint* (2018).
- [15] Rikard Laxhammar and Göran Falkman. 2014. Online Learning and Sequential Anomaly Detection in Trajectories. *IEEE Transactions on Pattern Analysis and Machine Intelligence* 36, 6 (2014), 1158–1173. doi:10.1109/TPAMI.2013.172
- [16] Zheng Li, Yue Zhao, Nicola Botta, et al. 2020. COPOD: Copula-Based Outlier Detection. In *IEEE ICDM*.
- [17] Zheng Li, Yue Zhao, Xiyang Hu, et al. 2023. ECOD: Unsupervised Outlier Detection Using Empirical Cumulative Distribution Functions. *IEEE TKDE* (2023).

- [18] Emaad Manzoor, Hemank Lamba, and Leman Akoglu. 2018. xStream: Outlier Detection in Feature-Evolving Data Streams. In *KDD*.
- [19] Yisroel Mirsky, Tomer Doitshman, Yuval Elovici, and Asaf Shabtai. 2018. Kitsune: An Ensemble of Autoencoders for Online Network Intrusion Detection. In *NDSS*.
- [20] Tomas Pevny. 2016. Loda: Lightweight on-line detector of anomalies. *Machine Learning* (2016).
- [21] Markus Ring, Sarah Wunderlich, Deniz Scheuring, Dieter Landes, and Andreas Hotho. 2019. A survey of network-based intrusion detection data sets. *Computers & Security* (2019).
- [22] Iman Sharafaldin, Arash Habibi Lashkari, and Ali A. Ghorbani. 2018. Toward Generating a New Intrusion Detection Dataset and Intrusion Traffic Characterization. In *ICISSP*.
- [23] Swee Chuan Tan, Kai Ming Ting, and Fei Tony Liu. 2011. Fast Anomaly Detection for Streaming Data. In *IJCAI*.
- [24] Gerrit J. J. van den Burg and Christopher K. I. Williams. 2020. An Evaluation of Change Point Detection Algorithms. *arXiv preprint arXiv:2003.06222* (2020).
- [25] Vladimir Vovk, Alexander Gammerman, and Glenn Shafer. 2005. Algorithmic Learning in a Random World. *Springer* (2005).